CO3 – ASSIGNMENT 3

C Programming Solutions

Name	B. Janani
Register Number	1912260492
Subject	C Programming
Assignment	CO3 – Assignment 3

This document contains seven C program solutions. Each program was compiled and executed, and its terminal output is included as an image.

Q1 – Searching a Library Catalogue

Program 1 — Complete C source code

```c
#include <stdio.h>

int steps;

int binary_search(const int a[], int n, int key) {
    int low = 0, high = n - 1;
    steps = 0;

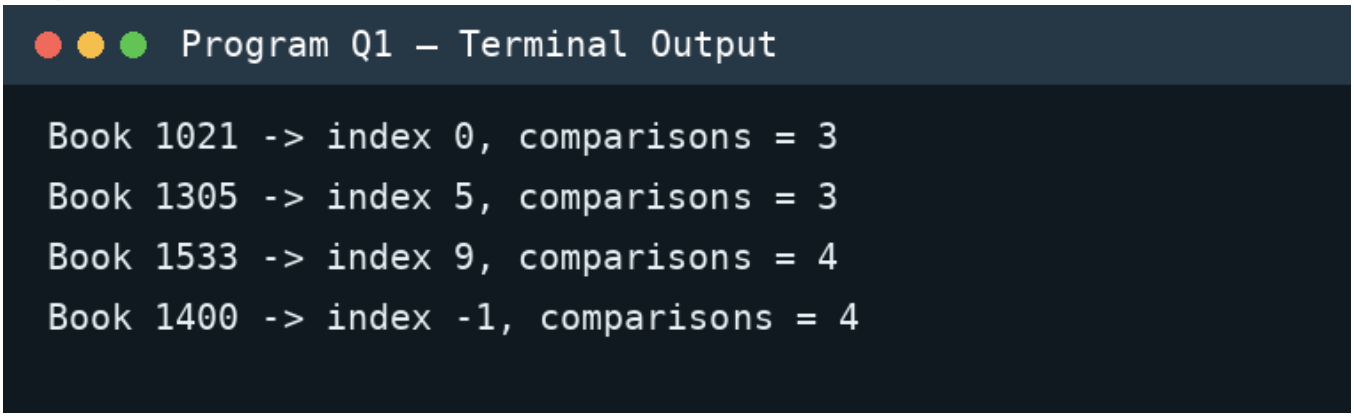
    while (low <= high) {
        int mid = low + (high - low) / 2;
        steps++;

        if (a[mid] == key)
            return mid;
        else if (a[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}

int main(void) {
    int accession[] = {1021, 1048, 1103, 1190, 1222,
                       1305, 1377, 1402, 1460, 1533};
    int n = sizeof(accession) / sizeof(accession[0]);
    int keys[] = {1021, 1305, 1533, 1400};

    for (int k = 0; k < 4; k++) {
        int pos = binary_search(accession, n, keys[k]);
        printf("Book %d -> index %d, comparisons = %d\n",
               keys[k], pos, steps);
    }
    return 0;
}
```

Output screenshot



Q2 – Sorting the Examination Hall Seating Register

Program 2 — Complete C source code

```c
#include <stdio.h>

void print_array(const int a[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
}

void bubble_sort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }
}

void selection_sort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;
        if (min != i) {
            int t = a[i]; a[i] = a[min]; a[min] = t;
        }
    }
}

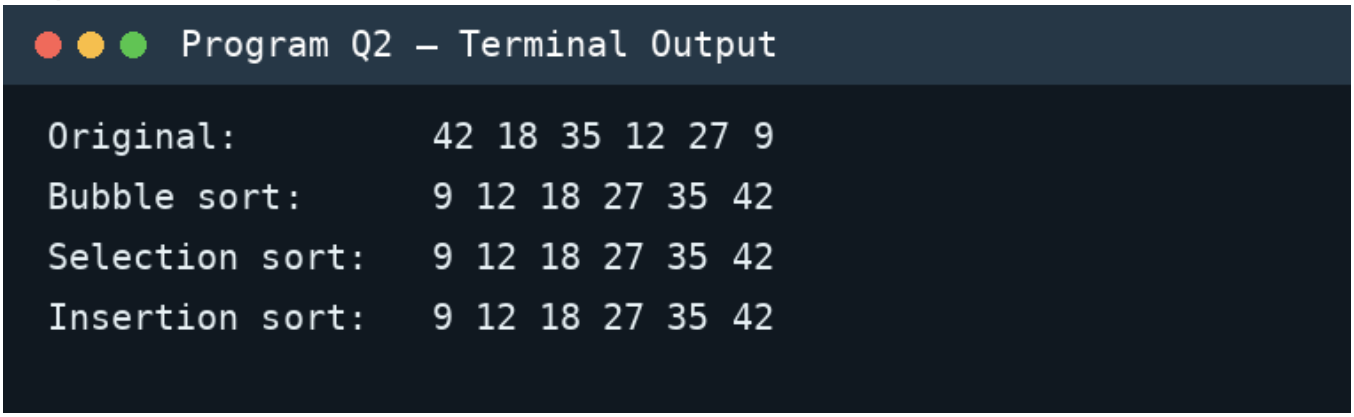
void insertion_sort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}

int main(void) {
    int x[] = {42, 18, 35, 12, 27, 9};
    int y[] = {42, 18, 35, 12, 27, 9};
    int z[] = {42, 18, 35, 12, 27, 9};
    int n = 6;

    bubble_sort(x, n);
    selection_sort(y, n);
    insertion_sort(z, n);

    printf("Original:         42 18 35 12 27 9\n");
    printf("Bubble sort:      "); print_array(x, n);
    printf("Selection sort:   "); print_array(y, n);
    printf("Insertion sort:   "); print_array(z, n);
    return 0;
}
```

Output screenshot



Q3 – Merging Two Merit Lists

Program 3 — Complete C source code

```c
#include <stdio.h>

/* Both input lists are in descending order. */
int merge(const int a[], int na, const int b[], int nb, int out[]) {
    int i = 0, j = 0, k = 0;

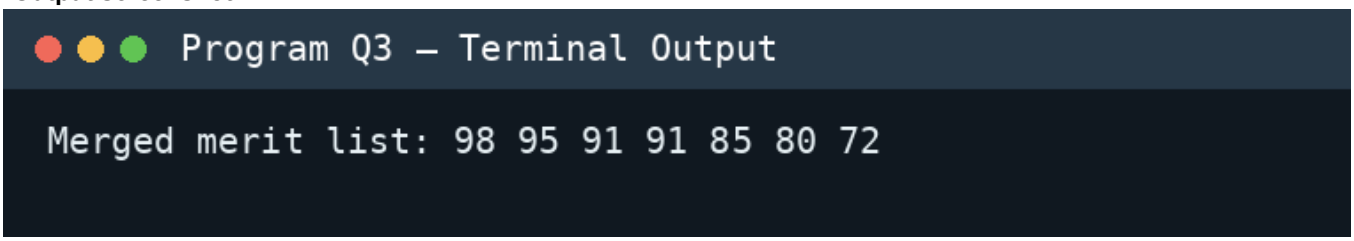
    while (i < na && j < nb) {
        if (a[i] >= b[j])
            out[k++] = a[i++];
        else
            out[k++] = b[j++];
    }
    while (i < na) out[k++] = a[i++];
    while (j < nb) out[k++] = b[j++];

    return k;
}

int main(void) {
    int campusA[] = {98, 91, 85, 72};
    int campusB[] = {95, 91, 80};
    int merged[7];

    int n = merge(campusA, 4, campusB, 3, merged);
    printf("Merged merit list: ");
    for (int i = 0; i < n; i++) printf("%d ", merged[i]);
    printf("\n");
    return 0;
}
```

Output screenshot



Q4 – Two-Dimensional Arrays: District Rainfall Records

Program 4 — Complete C source code

```
#include <stdio.h>

#define DISTRICTS 3
#define MONTHS 4

void annual_totals(const float rain[DISTRICTS][MONTHS],
                   float total[DISTRICTS]) {
    for (int i = 0; i < DISTRICTS; i++) {
        total[i] = 0;
        for (int j = 0; j < MONTHS; j++) total[i] += rain[i][j];
    }
}

void monthly_averages(const float rain[DISTRICTS][MONTHS],
                      float avg[MONTHS]) {
    for (int j = 0; j < MONTHS; j++) {
        float sum = 0;
        for (int i = 0; i < DISTRICTS; i++) sum += rain[i][j];
        avg[j] = sum / DISTRICTS;
    }
}

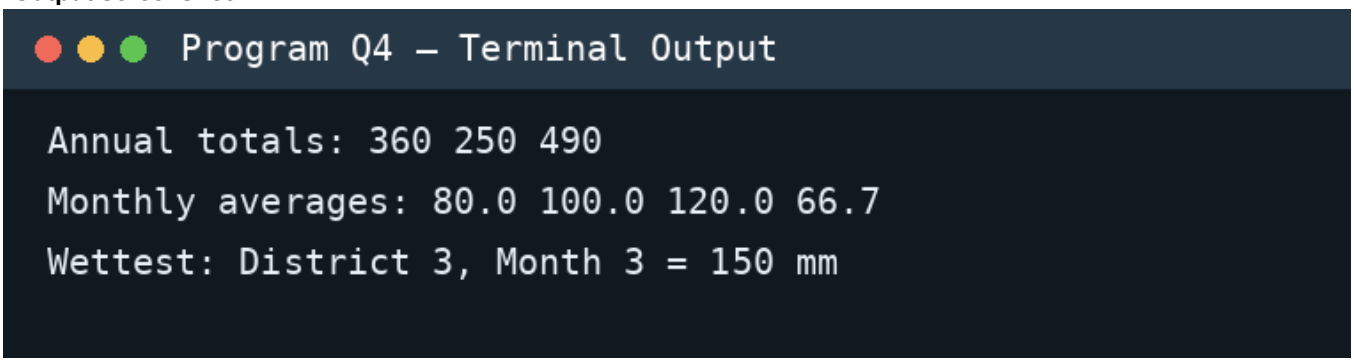
void find_wettest(const float rain[DISTRICTS][MONTHS],
                  int *district, int *month, float *maximum) {
    *maximum = rain[0][0]; *district = 0; *month = 0;
    for (int i = 0; i < DISTRICTS; i++)
        for (int j = 0; j < MONTHS; j++)
            if (rain[i][j] > *maximum) {
                *maximum = rain[i][j]; *district = i; *month = j;
            }
}

int main(void) {
    float rain[DISTRICTS][MONTHS] = {
        {80, 100, 120, 60},
        {50, 70, 90, 40},
        {110, 130, 150, 100}
    };
    float total[DISTRICTS], avg[MONTHS], maximum;
    int d, m;

    annual_totals(rain, total);
    monthly_averages(rain, avg);
    find_wettest(rain, &d, &m, &maximum);

    printf("Annual totals: ");
    for (int i = 0; i < DISTRICTS; i++) printf("%.0f ", total[i]);
    printf("\nMonthly averages: ");
    for (int j = 0; j < MONTHS; j++) printf("%.1f ", avg[j]);
    printf("\nWettest: District %d, Month %d = %.0f mm\n",
           d + 1, m + 1, maximum);
    return 0;
}
```

Output screenshot



Q5 – Processing Text Without the Library

Program 5 — Complete C source code

```c
#include <stdio.h>
#include <ctype.h>

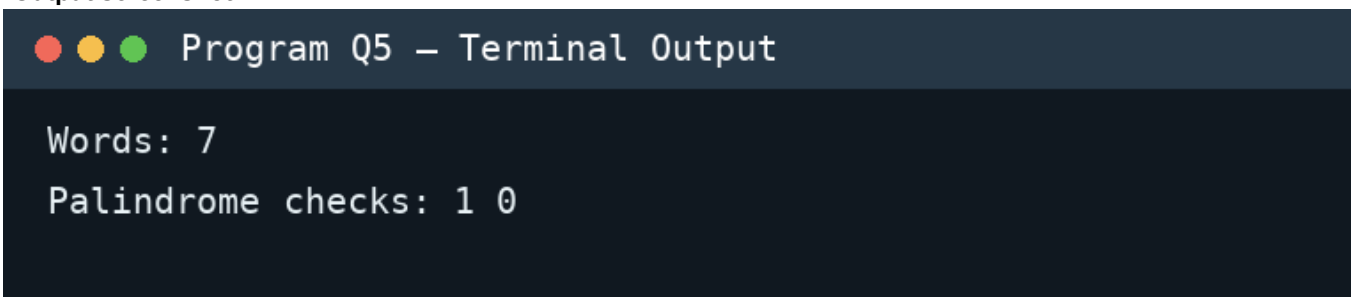
int word_count(const char s[]) {
    int count = 0, inside = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (s[i] != ' ' && s[i] != '\t' && s[i] != '\n') {
            if (!inside) count++;
            inside = 1;
        } else {
            inside = 0;
        }
    }
    return count;
}

int is_palindrome(const char s[]) {
    int left = 0, right = 0;
    while (s[right] != '\0') right++;
    right--;

    while (left < right) {
        while (left < right && !isalpha((unsigned char)s[left])) left++;
        while (left < right && !isalpha((unsigned char)s[right])) right--;
        if (tolower((unsigned char)s[left]) !=
            tolower((unsigned char)s[right])) return 0;
        left++; right--;
    }
    return 1;
}

int main(void) {
    printf("Words: %d\n", word_count(" Meet at the library at 5 PM "));
    printf("Palindrome checks: %d %d\n",
           is_palindrome("Madam, I'm Adam"), is_palindrome("Hello"));
    return 0;
}
```

Output screenshot



Q6 – A Directory of Names: Sorting and Searching Strings

Program 6 — Complete C source code

```c
#include <stdio.h>
#include <string.h>

#define N 5
#define LEN 30

void sort_names(char names[N][LEN]) {
    char temp[LEN];
    for (int i = 0; i < N - 1; i++) {
        for (int j = 0; j < N - 1 - i; j++) {
            if (strcmp(names[j], names[j + 1]) > 0) {
                strcpy(temp, names[j]);
                strcpy(names[j], names[j + 1]);
                strcpy(names[j + 1], temp);
            }
        }
    }
}

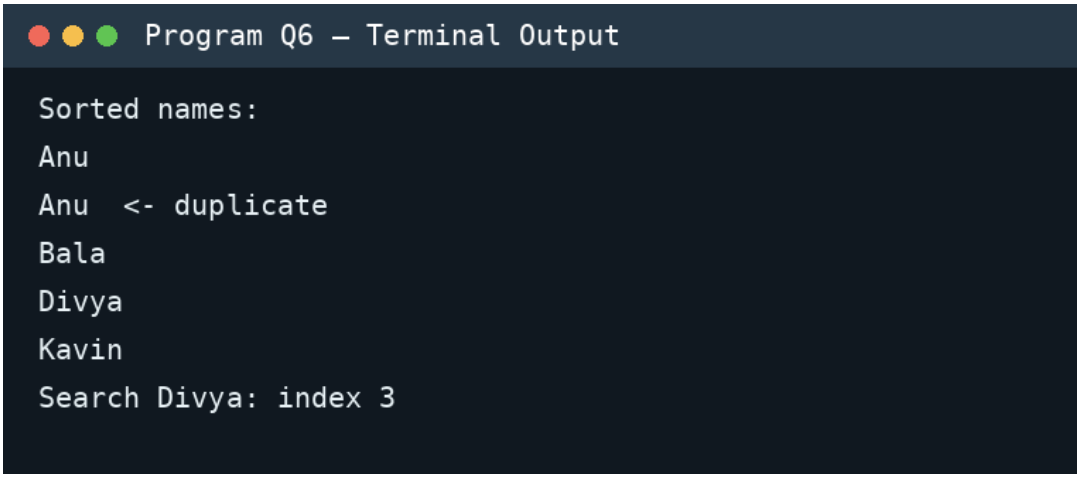
int binary_search(char names[N][LEN], const char key[]) {
    int low = 0, high = N - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        int cmp = strcmp(names[mid], key);
        if (cmp == 0) return mid;
        if (cmp < 0) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main(void) {
    char names[N][LEN] = {"Kavin", "Anu", "Bala", "Anu", "Divya"};
    sort_names(names);

    printf("Sorted names:\n");
    for (int i = 0; i < N; i++) {
        printf("%s", names[i]);
        if (i > 0 && strcmp(names[i], names[i - 1]) == 0)
            printf("  <- duplicate");
        printf("\n");
    }

    int pos = binary_search(names, "Divya");
    printf("Search Divya: index %d\n", pos);
    return 0;
}
```

Output screenshot



Q7 – Design an Election Counting Program from First Principles

Program 7 — Complete C source code

```c
#include <stdio.h>

#define CANDIDATES 4
#define VOTES 12

int main(void) {
    int votes[VOTES] = {1, 2, 1, 3, 2, 1, 4, 2, 5, 0, 3, 2};
    int count[CANDIDATES + 1] = {0};
    int valid = 0, invalid = 0, max = 0, winners = 0;

    for (int i = 0; i < VOTES; i++) {
        if (votes[i] >= 1 && votes[i] <= CANDIDATES) {
            count[votes[i]]++;
            valid++;
        } else {
            invalid++;
        }
    }

    printf("Valid votes: %d\nInvalid votes: %d\n", valid, invalid);
    printf("Candidate totals (descending):\n");
    for (int i = 1; i <= CANDIDATES; i++)
        printf("Candidate %d: %d\n", i, count[i]);

    for (int i = 1; i <= CANDIDATES; i++)
        if (count[i] > max) max = count[i];
    for (int i = 1; i <= CANDIDATES; i++)
        if (count[i] == max) winners++;

    if (winners > 1) {
        printf("Result: Tie (%d candidates with %d votes)\n", winners, max);
    } else {
        for (int i = 1; i <= CANDIDATES; i++)
            if (count[i] == max) printf("Winner: Candidate %d (%d votes)\n", i, max);
    }
    return 0;
}
```

Output screenshot

